

Predicting Survived and Killed Mutants

Alejandra Duque-Torres[†], Natia Doliashvili[†], Dietmar Pfahl[†], and Rudolf Ramler[‡]

[†]*Institute of Computer Science, University of Tartu, Tartu, Estonia*

E-mail: {alejandra.duque.torres, natia.doliashvili, dietmar.pfahl}@ut.ee

[‡]*Software Competence Center Hagenberg GmbH, Hagenberg, Austria*

E-mail: rudolf.ramler@scch.at

Abstract—Mutation Testing (MT) is a state-of-the-art technique for assessing test suite effectiveness. The MT principle is to inject variants, known as mutants, into the System Under Test (SUT). Then, the behaviour of the original SUT is compared to that of the mutated SUT when running the same test suite. If no difference in behaviour is observed, the mutant is said to have survived; otherwise, it is said to have been killed. Despite its strengths, the applicability of MT in practice has been limited by its high computational cost. To mitigate this problem, Predictive Mutation Testing (PMT) has been proposed. PMT uses a classification model based on features related to the mutated code and the test suite to predict the execution results of a mutant without actually executing it. In other words, PMT predicts whether a mutant will be killed or will survive. In previous studies, PMT has been evaluated on several projects in two application scenarios, involving cross-project and cross-version learning. The goal of our research is to investigate how well the proposed PMT method, which has been evaluated on Java, can be extended to other programming languages. For that purpose, we first replicated the previous study and then extended the PMT approach to a single C program. We used random forest classifiers as our supervised learning approach of choice. Our results indicate that PMT is able to predict the execution results of mutants with high accuracy. On the Java projects, we achieved Area Under Curve (AUC) values above 0.90 with a Prediction Error (PE) below 10%. On the C project, we achieved an AUC value above 0.90 with a PE below 1%. In our analyses we also investigated how sensitive the performance of PMT is to the set of selected features. In particular, we wanted to understand whether adding programming language specific features to a language independent core set of features significantly improve the performance of PMT. Our results are an indicator that, overall, PMT has potential to be applied across programming languages and is robust when dealing with imbalanced data.

Index Terms—Software testing, mutation testing, predictive mutation testing, machine learning, supervised learning

I. INTRODUCTION

Software testing is a fundamental stage in the software development life cycle, aiming at providing confidence that the software behaves in accordance with a specification or as expected by the end users. The effectiveness of software testing depends on the quality of its tests. However, evaluating the quality of a test suite is not a trivial task. One of the related challenges is the constantly increasing size and complexity of software and, accordingly, the increasing size and complexity of the corresponding test suites. For instance, Google reported that they integrate and test more than 13K projects on an average day with code committed about every second, requiring 800K builds and 150 million test runs [1]. Several

techniques have been developed to assess test suite quality including *Mutation Testing* (MT) [2], *Code Coverage* [3], and *Error Seeding* [4]. In particular, MT is currently the state-of-the-art technique for assessing test suite quality [5], [6]. MT is a fault-based testing technique that provides a test suite metric called the *mutation score*. This technique has three major steps. First, from a correct System Under Test (SUT) slightly altered versions are created by automatically introducing faults into code lines. Such faults are known as *mutants*. Second, it is checked whether the existing test suite (consisting only of tests passing on the original SUT) can distinguish the mutants from the original SUT. If a mutant is been detected by a failing test, then the mutant is labelled as *killed mutant*. If all tests pass on a mutant it is labelled as *survived mutant*. Finally, the relationship between the number of killed mutants and the number of all mutants is recorded as the *mutation score*. The desired mutation score is 1, i.e., all mutants are killed by the test suite. If certain mutants have not been killed, the tests have to be improved or new ones have to be added to the test suite to strengthen its effectiveness.

A disadvantage of MT is that it is an extremely costly and time-consuming technique since each mutant needs to be executed against the test suite. This heavily limits the scalability of MT, in particular in environments aiming to implement continuous integration and testing [7]. In addition, problems related to the difficulty of automatically detecting (and filtering out) equivalent mutants, i.e., mutants that have the same behaviour as the original SUT, limit the robustness of MT. Several approaches have been proposed to overcome those problems. For instance, the application of selective mutation [8], mutant sampling [9], and minimum mutant sets [10], [11]. While these techniques can reduce the cost of MT to some degree, the test suite still needs to be executed to detect the mutants. To address the problem of long and costly runs of existing tests against generated mutants, the use of Machine Learning (ML) techniques has been suggested to predict execution results without the need to actually execute the test suite on the mutants.

Predictive Mutation Testing (PMT) [12]–[14] is a recently proposed method that uses ML to predict the outcomes of MT (i.e., whether a mutant is killed or survived) without test suite execution. PMT collects easy-to-access features of the SUT, the test suite, and the outcomes of past MT rounds. Then, an ML model is trained to predict the outcome of MT on a new SUT with a corresponding test suite. The goal is to enable the

trained model to predict whether or not the mutants of a new SUT are killed without actually executing the corresponding test suite. Motivated by the study *Predictive Mutation Testing* published by Zhang *et al.* [12], [13], we wanted to investigate the potential of PMT. In particular, we were interested in understanding whether the results achieved by Zhang *et al.* on Java code can be transferred to other programming languages. More specifically, we asked ourselves whether we can predict MT results without mutant execution for real-world projects where the code is written in the C programming language at the same performance level as achieved by Zhang *et al.* [12], [13] on Java code.

We tackle our goal in two steps by answering the following research questions:

- **RQ1:** Is it possible to replicate the results of the paper “*Predictive Mutation Testing*” by Zhang *et al.* [12], [13] using the same Java projects?
- **RQ2:** How can we transfer the PMT approach across different programming languages?

In the context of our study, we focus on the transfer of PMT from the open source Java projects used by Zhang *et al.* to C projects. In particular, we are interested in understanding whether and how it is possible to create similar ML models for a system implemented in C, using a related but adapted set of features.

The rest of the paper is structured as follows. In Section II, we elaborate on the concept of MT and PMT. In Section III, we describe the PMT methodology and its setup in different configurations and using different modelling tools. In Section IV, we present the answers to our research questions. We discuss our results in Section V. Finally, we conclude the paper in Section VI.

II. BACKGROUND

In this section, we give a brief background on the main concepts used in our paper. We start with the definition of MT in Section II-A. Then, in Section II-B, we introduce PMT.

A. Mutation Testing

Since MT was introduced in the 1970s, it has been widely studied with increasing interest in recent years [6], [15]. Several studies have shown MT as a strong method for identifying high-quality test suites at different software testing levels (unit, integration, and specification) and for different programming languages. Furthermore, MT has been shown to be suitable for guiding test generation [16], [17], simulating real faults in software testing experimentation [5], [18], and localising faults [19], [20]. Overall, MT is a fault-based testing technique that can be used to define a test suite quality metric, i.e., the so-called mutation score. MT consists of three main steps:

1. **Mutant Generation:** From a correct program or SUT, several faulty versions, the so-called *mutants*, are automatically generated by introducing small changes into the code of the original SUT. There exist different ways of creating mutants, e.g., changing variable names,

changing operands and operators, removing or adding code lines, and so on.

2. **Mutant Execution:** To check whether the test suite can detect the mutants generated in the previous step, the test suite is run on each mutant. If a test in the test suite fails, the mutant has been *killed*, if no test fails, the mutant *survived*.
3. **Mutation Score Calculation:** The mutation score is calculated by dividing the amount of killed mutants by the total amount of mutants.

Ideally all mutants are killed by the existing test suite (i.e., the mutation score equals 1). Numerous tools for generating mutants have been developed for various programming languages [15]. Examples include *MuJava* [21], *Major* [22] and *PIT* [23] for Java, and *Milu* [24] for the C language. Typically, these tools offer various types of mutation operators (predefined rules for making changes to the original SUT to create mutants), support for reducing the number of (redundant) mutants and avoiding equivalent mutants (i.e., mutants that are syntactically different from the original SUT but show identical behaviour), and optimisations for speeding up the mutant generation and test process.

The high computational costs are considered as one of the main factors hindering the wider adoption of MT within industry [25]. In the mutant execution step, MT requires running the tests from the test suite against each of the generated mutants. In real-world projects, a large number of mutants can be generated. For example, as reported in [26], MT applied for a safety-critical embedded system with 60 KLOC produced 75 043 mutants. Thus, the maximum possible number of test runs is usually huge. It is equal to the number of tests times the number of mutants (full kill matrix). Even though that in practice the number of test runs can be significantly reduced when tests are executed until one of them eventually kills the mutant, the remaining number of test executions is usually still very high. This is especially true, if there is a large number of surviving mutants, as for each of them all the tests of the test suite have to be run.

Among the various techniques proposed to reduce execution time and costs in MT [27], the use of ML has been suggested for predicting killed or survived mutants without the need of running the test suite once for each mutant.

B. Predictive Mutation Testing

PMT is a new predictive method to help reduce the cost of MT, originally proposed by Zhang *et al.* [12], [13]. PMT is an approach that predicts the results of mutant execution without actually executing the mutants against the test suite. More precisely, this approach takes advantage of ML by training a classification model using a set of features related to mutants and tests. Once the PMT model is trained, it can be used to predict whether an existing test suite will kill or not a generated mutant without the need to run the test suite. Zhang *et al.* [12], [13] evaluated their PMT approach on 163 open source Java projects. Their results indicate that PMT offers a dramatic reduction of execution cost in exchange for

only small accuracy losses. Observed efficiency improvements were more than 150X. Two evaluation scenarios were applied: *cross-version* and *cross-project*. In the cross-version scenario, results from MT on a particular version n of one of nine evaluation projects were predicted with a model trained on data from a previous version $n-1$ of the same project. In the cross-version scenario, predictions were made with models built using data from different projects. PMT performed well in both scenarios. The results show that PMT is very effective under the cross-version scenario. It achieves over 0.90 for the performance measures (*i.e.*, precision, recall, F-measure and AUC). Under the cross-project scenario PMT achieves over 0.85 AUC and lower than 15% error on predicting mutation scores. Overall, PMT was clearly more efficient than traditional mutation testing, where each of the mutants is executed against the test suite.

The proposed PMT approach consists of three main steps: *i)* Feature (or attribute) selection, *ii)* Collection of training instances per attribute, and *iii)* Building the ML classification model. Features are chosen following the categories of PIE theory [28], with the three letters of the acronym staying for Propagation, Infection, and Execution. *Execution* features are features related to code lines in the SUT or mutants that are executed when running the test suite. *Infection* features are features related to program states of mutants that are affected when running the test suite. *Propagation* features are features related to infected program states that are propagated so that the output of the mutant is affected in such a way that it differs from the output of the original SUT. We discuss the feature selection in detail in Section III-B

To collect training instances, Zhang *et al.* [12], [13] used three different tools, *PIT* [29], *Major* [22], and *Cobertura*. The tools Major and PIT were used to generate and execute mutants. Cobertura was used to collect data of code coverage related features. Data on infection features were collected directly from the MT tools. For extracting propagation features, the authors developed custom tools. To develop the classification model, supervised learning algorithms were applied from the ML environment *Weka* [30], specifically, *Random Forest*, *Naïve Bayes*, *Support Vector Machine* (SVM), and *J48*.

III. METHODOLOGY

In this section, we provide a detailed description of the data used in our study as well as the experimental design. First, we describe the dataset used (Section III-A). Second, we describe the feature selection (Section III-B). Third, we briefly explain what data pre-processing we implemented (Section III-C). Finally, we present the performance measures used in this paper (Section III-D).

A. Datasets

In our study, we took the same set of Java base projects¹ as Zhang *et al.* [12], [13]: *Java APNS* (apns), *AssertJ* (assj), *Joda-Time* (joda), *Linear Algebra for Java* (lafj), *Apache Commons*

Lang (lang), *Message Pack for Java* (msg), *UAA* (uaa), *Vraptor* (vrap), and *Wire Mobile Protocol Buffers* (wire). This sample is a subset of the 163 projects used in the cross-project experiments described in [13]. For more information about the nature and characteristics of the nine projects, we refer the reader to [12], [13].

In addition to the nine Java projects, in our study, we also used an industry project written in C. This C project originates from an analysis of MT in a safety-critical embedded system performed by Ramler *et al.* [26]. The embedded software controls the electrical and mechanical components of a mechatronic system. It consists of a real-time operating system, platform-specific libraries, and an application consisting of 30 domain-specific components. The whole system was written in the C programming language. The application has about 60 000 lines of code (LOC). Component sizes range between 400 and 7000 LOC. Mutation testing produced 75 043 mutants of which 27 158 survived test execution. For more information about the original study on using MT in a safety-critical industry project, its design, implementation, and evaluation, we refer the reader to [26].

B. Feature Selection

Feature selection is done using the PIE analysis technique. The idea behind PIE analysis is not to detect if there is a fault in the program, but instead, it identifies locations in the program where faults are likely to stay undetected when executing a test suite. PIE establishes three conditions that are necessary for deciding whether a mutant has been killed. The first condition is *Execution*: a mutated statement in the SUT needs to be executed by at least one test. The second condition is *Infection*: execution of the mutated statement affects the program state, a pre-condition that the mutant can be identified. The third condition is *Propagation*: the infected program state returns output that is distinct from the output of the original SUT. The values of the above-mentioned conditions can be obtained for each mutant.

The idea behind PMT is to build a model that predicts whether a mutant is killed by analysing these conditions. For this purpose, it is necessary to identify sets of features representing each of the three condition types. In our study, we used four categories of features, three categories corresponding to each of the PIE condition plus one additional category characterising the test suite used to execute the SUT and its mutated versions.

1. *Execution features* – The execution feature category consists of features that are related to the execution of a mutated statement. These features signal whether a mutated part of the program was executed.
2. *Infection features* – The infection feature category identifies features that are related changes in program state induced by the mutation of the code. Since changes in the program state depend on changes made in a program statement of the original SUT, we need to define features characterising the type and nature of the mutation made.

¹<https://github.com/sei-pku/PredictiveMutationTesting>

TABLE I: Features of *Java* and *C* projects

Category	Feature	Description	Language	
			Java	C
Execution	<i>numExecuteCovered</i>	Indicates how many times a mutated line of the program is executed by the test suite	✓	–
	<i>numTestCovered</i>	Indicates how many tests from the test suite reach a mutated method of the program	✓	✓
Infection	<i>typeStatement</i>	Indicates what type of statement was mutated (<i>e.g.</i> , conditional or return)	✓	–
	<i>typeOperator</i>	Indicates what kind of mutation was done on the statement	✓	✓
Propagation	<i>McCabeCyclomatic_Complex</i>	Indicates exactly what it says: McCabe complexity of the mutated statement	✓	✓
	<i>Method_LOC</i>	Describes the number of lines of code in the mutated method	✓	✓
	<i>Depth_InheritanceTree</i>	Shows what is the length from mutated class to the root class	✓	–
	<i>Nested_BlockDepth</i>	Refers to the depth of nested blocks in the mutated method	✓	–
	<i>Number_Children (Ce)</i>	Describes how many subclasses the mutated class has	✓	–
	<i>Afferent Coupling (Ca)</i>	Indicates how many classes outside of the mutated package depend on classes inside the package	✓	–
	<i>Instability</i>	Is calculated using the previous two features: $Ce/(Ce+Ca)$	✓	–
	<i>Mutation</i>	Indicates actual replacement for the mutated source code element	–	✓
	<i>Branches</i>	Describes the number of branches, for example, the number of if statements in the tested function	–	✓
	<i>Loops</i>	Indicates the number of loops, for example, the number of for statements in the tested function	–	✓
	<i>Maintainability</i>	Refers to the maintainability index	–	✓
	<i>Operands</i>	Indicates the number of total operands	–	✓
	<i>Unique_operands</i>	Indicates the number of distinct operands	–	✓
	<i>Unique_operators</i>	Indicates the number of distinct operators	–	✓
	<i>Volume</i>	Describes Halstead complexity	–	✓
	<i>Lines</i>	Indicates the number of source code lines with blank lines.	–	✓
	<i>Operators</i>	Indicates the number of total operators	–	✓
Others	<i>numMutantAssertion</i>	Indicates the number of assertions in the test methods that cover each method	✓	✓
	<i>numClassAssertion</i>	Indicates the number of assertions in the test class that covers the mutated class	✓	✓
	<i>typeReturn</i>	Indicates the return type of the mutated method	✓	✓
	<i>cfileId</i>	Tested C file	–	✓
	<i>MethodId</i>	Tested function or procedure in the C file	–	✓
	<i>lineInMethod</i>	Line location of the mutated source code element within code unit (function or procedure)	–	✓
	<i>Line</i>	Line location of the mutated source code element	–	✓
	<i>sloc</i>	Number of source code lines without blank lines (source code metric)	–	✓
	<i>Location</i>	Represents location hierarchy: package and class that mutant is located in	–	✓
	<i>Column</i>	Column location of the mutated source code element	–	✓

3. *Propagation features* – The propagation feature category contains features that are related to the potential of propagating an infected program state to observable behaviour. This category assembles program characteristics such as complexity. If a program is complex, then there is a higher possibility that the program state produced by the mutated statement alters the program's output.
4. *Other features* – This additional category of features characterises the test suite. It complements the feature sets corresponding to the PIE conditions.

Table I presents the list of features defined in each category for both *Java* and *C* projects. Which feature has been used in the analyses related to *Java* and *C* programs, respectively, is indicated by a tick in either of the last two columns. Features that are common for both *Java* and *C* programs are printed in bold letters.

C. Preprocessing Data

In MT, usually, the number of killed and survived mutants is unbalanced. Typically, one can expect more killed mutants

than survived mutants. The problem with unbalanced data in ML is that it negatively affects the classifier's performance, especially for those patterns that belong to the less represented class(es). To overcome the problems induced by unbalanced data, several methods have been proposed. One of the strategies which have shown to be effective is under-sampling. Especially, random under-sampling for the majority class. The idea behind random under-sampling is to remove instances of the majority class randomly and uniformly. However, this approach can lead to unwanted information loss, since one may remove instances that are inducing strong signals. However, if instances of the majority class are similar, this method typically shows good results.

Another good strategy for handling unbalanced data is cost-sensitive learning. Cost-sensitive learning assigns a higher cost to the misclassification of members in the minority class, thus minimising the total cost. For instance, Random Forest is designed to minimise the overall error rate. To do so, it focuses on maximising the accuracy of majority class predictions and, as a consequence, results in lower accuracy predictions for

the minority class. Weighted random forest assigns weights to both classes with the minority class having a larger weight. This results in higher misclassification cost for the minority class and thus helps improve prediction performance for this class when trying to minimise this cost.

PMT uses a *Supervised Learning* (SL) approach where a classification model is trained on a set labelled data, i.e., the training data. Once sufficiently well trained, the classification model is used for making predictions. In PMT, the training data consists of the mutants which already have been executed against the test suite of previous project versions and the associated feature values. The model makes predictions on new mutants of the current project version and classifies them as either *killed* or *survived*. This kind of prediction is also known as a binary classification.

In our study, we used *Random Forest* (RF) as the SL approach of choice. RF uses an ensemble learning algorithm [31]. It consists of a set of Decision Trees (DT) that operate together. The DT algorithm works by breaking down a data set into smaller and smaller subsets based on the similarity of their features. Then, Random Forest creates a large number of such DTs. In order to make a prediction on a test instance, it aggregates votes of each individual DT and predicts the class with the most votes. An important pre-requisite for a well-functioning RF is that the DTs have a low correlation among themselves. The reason for this is that while some trees make bad predictions many other trees make good predictions, and thus, as a group, they will be able to make a good prediction.

D. Performance Measures

In our study we used performance measures derived from the *Confusion Matrix*. Let A and A' denote a classification (e.g., a killed mutant), then each reference standard measure is expressed as a function of the *Confusion Matrix* defined as follows:

True Positive (TP): The actual class of a case was A and the predicted class was A . This represents a successful prediction.

True Negative (TN): The actual class of a case was A' and the predicted class was A' . This represents a successful prediction.

False Positive (FP): The actual class of a case was A' and the predicted class was A . This represents an unsuccessful prediction.

False Negative (FN): The actual class of a case was A and the predicted class was A' . This represents an unsuccessful prediction.

Accuracy is the ratio of successful predictions made to both classes and expressed as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Precision (or positive predictive value) is the ratio of correct predictions made for class A and is shown in Equation (2):

$$Precision = \frac{TP}{TP + FP}. \quad (2)$$

The true positive rate (or recall) is the ratio of successful predictions made to cases of class A .

$$Recall = \frac{TP}{TP + FN}. \quad (3)$$

The *f-measure* statistic (or F1 score) considers both the DR and precision of a classifier to measure its quality:

$$f\text{-measure} = 2 \times \frac{Recall \times Precision}{Recall + Precision}. \quad (4)$$

In addition to the aforementioned performance measurements, we provide the *Area Under Curve* (AUC) and *Prediction Error* (PE) measurements. The AUC is the area under the curve of the plot False Positive Rate (FPR) *vs* True Positive Rate (TPR) at different points in $[0, 1]$. PE is the difference between the mutation score calculated on test data and the mutation score calculated on the predicted results of mutants of test data.

IV. RESULTS

We first evaluated the PMT approach using the nine Java base projects adopted from Zhang *et al.* [12], [13] under the two scenarios, i.e., *cross-version* and *cross-project*. Different to Zhang *et al.*, who used *Weka* [30] to build the classification model, we used the Python *scikit-learn* library [32]. Then we evaluated the PMT approach on a large C project.

Section IV-A1 presents the results obtained for the nine Java projects in the *cross-project* scenario. Section IV-A2 presents the results obtained in the *cross-version* scenario. Then, in Section IV-B we present the results obtained when applying PMT on the C project.

A. RQ1 – Performance of PMT for Java Projects using Python

1) *Cross-project scenario*: The cross-project scenario uses the newest version of each project as test data after training the classification model using the newest versions of the other projects. We compared the results of Random Forest obtained by Zhang *et al.* [12], [13] who used *Weka* [30] with default parameter settings with our results when using Python *scikit-learn* library [32] with default parameter settings. In addition, we tuned Python to run RF with parameter settings equivalent to the default settings in *Weka*. Table II summarises the results obtained. To distinguish our results from the results obtained by Zhang *et al.* [12], [13], and the results obtained after tuning the parameters, the names of corresponding projects have been marked with letters Z and T , respectively. Accordingly, the simple *project* name refers to the results achieved with Random Forest using Python with default parameter settings, the name *project-T* refers to results achieved after adjusting the Random Forest parameter settings in Python to match those used in the *Weka* tool. Finally, *project-Z* refers to the results obtained by Zhang *et al.* [12] who used the *Weka* tool.

From Table II we see that in most cases, classification models used by Zhang *et al.* [12], [13] have the best performance. Only in the case of project *wire* the classification model using Python with parameter tuning are better than the original

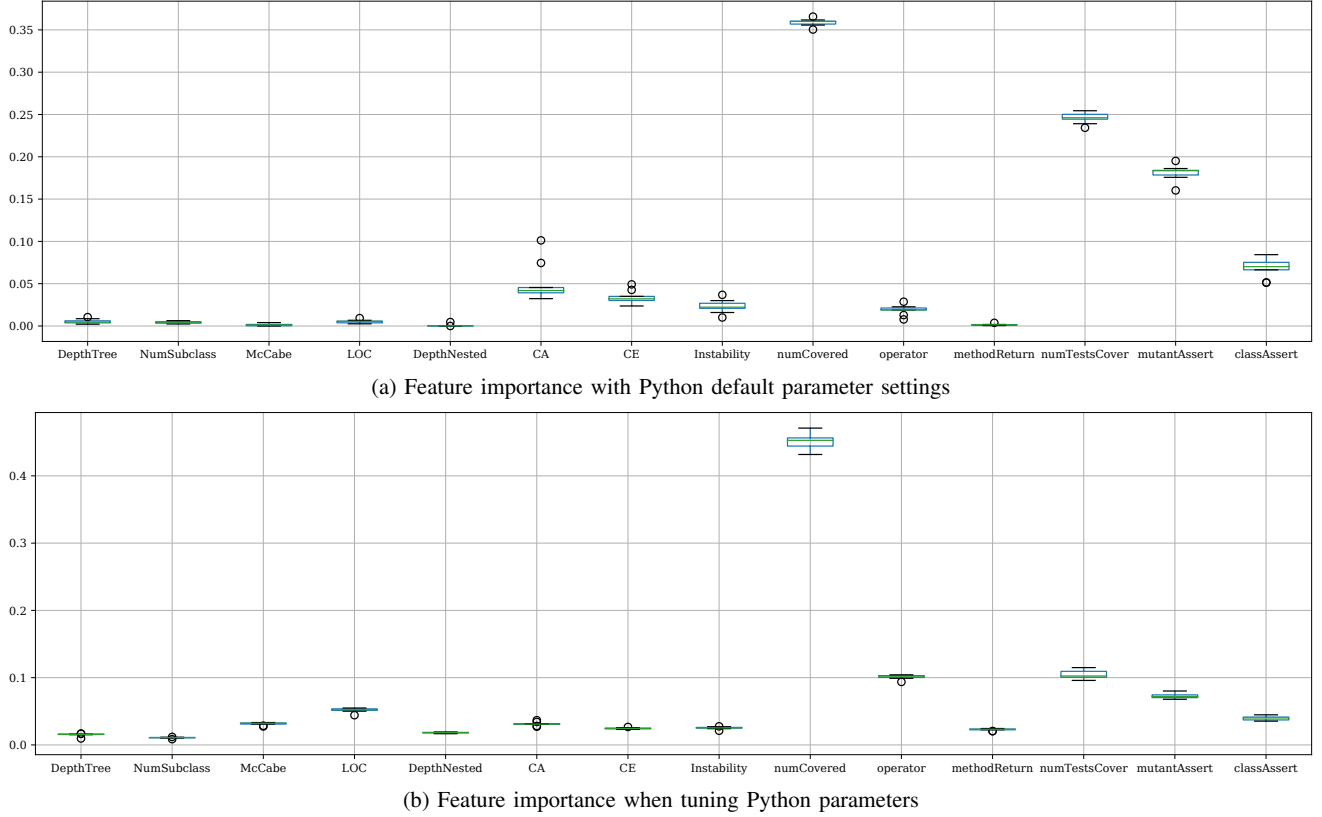


Fig. 1: Feature importance comparison between Python default parameter settings and tuning the Python parameters to match those in the *Weka* tool

results. Classification models developed with Python default parameter settings show consistently the lowest performance. Overall, however, the differences between the three approaches are not large.

Figure 1 shows the features importance comparison between models generated with Python default parameters and models generated with Python and parameters tuned to match those of *Weka*. In both Figures 1(a) and 1(b) the feature *numCovered* has the highest importance. The feature *numCovered* indicates how many times a mutated line of the program is executed by the test suite. Overall, the importance of the features differ subtly depending on the parameters.

2) *Cross-version scenario*: The cross-version scenario uses already executed mutants of earlier versions of the same program to build the classification model. More precisely, in the *cross-version* scenario the classification model uses an immediate previous version of the program as training data and the current version as test data. For instance, project *apns* has five different versions (V). Then, *apns*- V_0 is used as training data, and *apns*- V_1 (in this case, the current version) is used as test data. Due to space constraint, in this paper, we only show the *apns* and *assj* results. However, all the results and their tables and plots can be found in our GitHub

repository². Tables III and IV summarises the result obtained. To distinguish our results from the results obtained by Zhang *et al.* [12], [13], and the results obtained by us, the names of the projects have been changed. Accordingly, *apns*-Z and *assj*-Z refer to the results published by Zhang *et al.* [12].

From Tables III and IV one sees that all prediction errors are below 4% and most of the performance measures are above 0.9. These results are very close to the results obtained by Zhang *et al.* [12] for both Java projects. More precisely, the F-measure values differ at most by 0.03 and at least by 0.001, AUC values differ at most by 0.1 and at least by 0.031. Prediction errors are very close too. Overall, this means that the replication of results for the *cross-version* scenario was successful. Small differences between values are expected, because the conditions of the experiment were not exactly the same. In particular, different tools were used for building the classification models.

Figure 2 shows the feature importance of *cross-version* scenario. In comparison with the *cross-project* scenario from Figures 1(a) and 1(b), the *cross-version* scenario keeps the feature *numCovered* as main feature. Overall, as Figure 2 shows, the *cross-version* scenario shows similar feature importance as the *cross-project* scenario in Figure 1(b).

²<https://github.com/aduquet/Predicting-Survived-and-Killed-Mutants>

We also analysed the impact of version intervals on prediction performance. Table V provides the results for the Java projects *apns* and *assj*. In this analysis, the first version of each project is used as the training set, e.g., V_0 . The other versions are used as test sets. Similar to the previous results, the *AUC* values are all above 0.90, all the other metrics values are above 0.82, and prediction error values are below 9%. Note that as version difference increases the *AUC* value decreases. This is an effect of changes between the versions, i.e., the more changes are made to the project the more different the versions are. From Table V we can deduce that PMT performs quite well even when versions are distant from each other. This means that a model can be trained on a version and used to make predictions on the several following versions with high accuracy.

B. RQ2 – Transfer of PMT from Java to C

This section presents an evaluation of PMT applied to the C project. The same approach for PMT has been used as in the previous experiments. However, the C project differs from the Java projects in several ways, besides the programming language. With about 60 000 LOC the C project is about two times the size of the largest Java project (Joda-Time) and the number of generated mutants for the C code (75 043) is more than three times larger than the maximum number of mutants obtained for a Java project (Joda-Time). The higher ratio of mutants to LOC for the C project is likely due to the coding style used in embedded development (e.g., frequent use of constant values) as well as the application of a different mutation tool (Milu). The different tools for mutation generation and testing also resulted in a different set of features used in PMT. Only eight of the 24 features extracted from the C project were shared with the 15 features used in the Java projects. Furthermore, the studied C project is a safety-critical industry system. In this project, the safety standards required 100% MCDC coverage of the entire code base. So for each source code line there is at least one test case that covers that line, in contrast to the Java projects, where parts of the code were not covered by tests at all and therefore none of the corresponding mutants could be killed.

1) *Performance measurement*: First, all the available 24 features (see full list of features in Table I) were used to build the RF classification model. As the C project has one version only, the data was divided into training and test sets using different splits (ranging from 90:10 to 50:50) to evaluate the performance of the model. The results are shown in Table VI. One can see that the performance level is comparable to that when applying PMT to Java. Also, the split between training and test data doesn't seem to have much impact on the performance.

Figure 3 orders features for the C project according to their importance in the predictions. To check whether a reduction of the number of features in model building, i.e., removing the less important features, impacts model performance, we compared the performance when using different numbers of features. Table VII summarises the result obtained. To build the classification models, the data was split into 80% for training and 20% for testing. First, we used all the 24 features. Second, we used the 10 features with highest importance, that is, *Column*, *Line*, *lineInMethod*, *Mutation*, *cfileId*, *operators*, *uniqOperators*, *TypeOperator*, *volumen*, and *operands*. Then, *volumen* and *operands* were dropped, and then we used only 8 features with the highest importance for training. Finally, we made use of only the 5 most important features, that is, *Column*, *Line*, *lineInMethod*, *Mutation*, and *cfileId*.

From Table VII it is evident that almost the same performance is obtained using only a few important features. This finding can greatly help PMT since extracting only 10 or even 5 features is much easier than collecting the data required for all 24 features. In addition, four out of five features with high importance, i.e., *Column*, *Line*, *lineInMethod*, and *Mutation*, are location features. Location feature are very easy to collect as only the source code is required, unlike test related features.

2) *Common features between C and Java projects*: The evaluated C and Java projects have eight features in common (features printed in bold in Table I). We first build a PMT model using the set of shared features, to build a cross-project prediction model independent from the programming language. Using RF, we trained a model on the latest versions

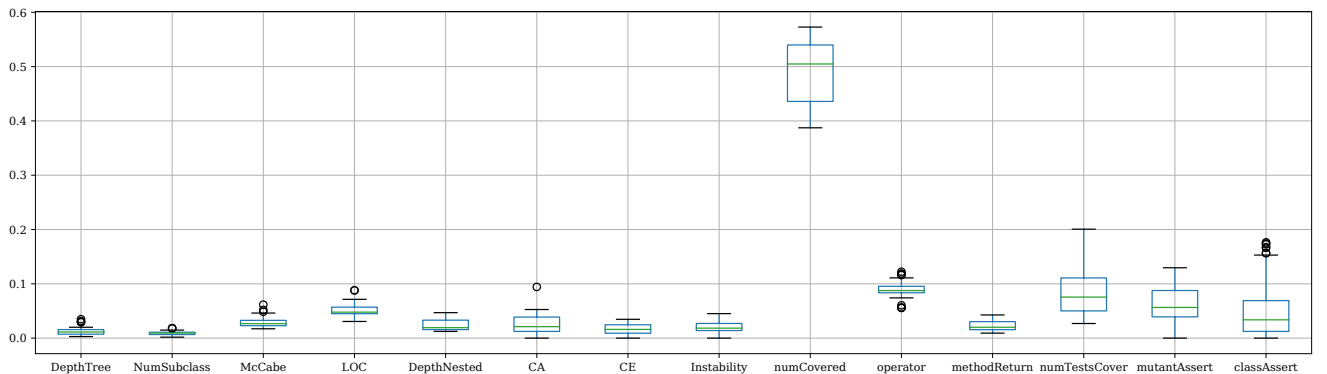


Fig. 2: Feature importance Cross-version scenario for Java projects

of each Java project (i.e., *cross-project scenario*) and the C project using Python with default parameter settings. For each Java project and the C project, the data was divided into training and test sets. The outcome of the models is presented in Table VIII. From the results, it is evident that the performance of the Java projects is comparable to that of the C project performance.

In addition, the performance of PMT for the C project was compared to those of the Java projects in a single version scenario. Therefore, prediction models were built from all versions of Java projects, again, only using the features shared between the C and the Java projects.

TABLE II: Performance comparison between Weka and Python

Proj	Performance measurements				
	Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
<i>apns</i>	0.752	0.831	0.825	0.826	0.169
<i>apns-T</i>	0.872	0.914	0.914	0.912	0.061
<i>apns-Z</i>	0.897	0.884	0.884	0.935	0.087
<i>assj</i>	0.903	0.905	0.889	0.711	0.085
<i>assj-T</i>	0.923	0.920	0.912	0.773	0.059
<i>assj-Z</i>	0.916	0.919	0.911	0.839	0.056
<i>joda</i>	0.817	0.835	0.807	0.690	0.164
<i>joda-T</i>	0.819	0.833	0.807	0.692	0.156
<i>joda-Z</i>	0.891	0.877	0.87	0.898	0.112
<i>lafj</i>	0.821	0.859	0.848	0.801	0.141
<i>lafj-T</i>	0.826	0.858	0.849	0.804	0.128
<i>lafj-Z</i>	0.888	0.876	0.869	0.876	0.107
<i>lang</i>	0.861	0.867	0.832	0.622	0.133
<i>lang-T</i>	0.864	0.858	0.829	0.629	0.113
<i>lang-Z</i>	0.904	0.901	0.896	0.896	0.071
<i>msg</i>	0.829	0.882	0.879	0.864	0.117
<i>msg-T</i>	0.829	0.881	0.877	0.863	0.114
<i>msg-Z</i>	0.919	0.909	0.908	0.924	0.08
<i>uaa</i>	0.719	0.824	0.821	0.839	0.176
<i>uaa-T</i>	0.721	0.815	0.814	0.829	0.151
<i>uaa-Z</i>	0.924	0.906	0.91	0.957	0.073
<i>wrap</i>	0.830	0.858	0.844	0.772	0.141
<i>wrap-T</i>	0.874	0.884	0.879	0.830	0.077
<i>wrap-Z</i>	0.9	0.895	0.89	0.882	0.08
<i>wire</i>	0.846	0.879	0.871	0.823	0.118
<i>wire-T</i>	0.878	0.801	0.804	0.796	0.049
<i>wire-Z</i>	0.858	0.853	0.855	0.919	0.034

[±]Precision, [‡]f-measure, [†]Prediction Error [%]

TABLE III: Performance comparison for the *apns* project under the *cross-version* scenario

Version		Proj	Performance measurements				
Tr [†]	Ts [◊]		Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
V ₀	V ₁	<i>apns</i>	0.928	0.947	0.947	0.944	0.029
		<i>apns-Z</i>	0.949	0.949	0.949	0.988	0.020
V ₁	V ₂	<i>apns</i>	0.865	0.924	0.925	0.930	0.050
		<i>apns-Z</i>	0.915	0.914	0.914	0.983	0.021
V ₂	V ₃	<i>apns</i>	0.956	0.966	0.966	0.966	0.004
		<i>apns-Z</i>	0.966	0.966	0.966	0.997	0.004
V ₃	V ₄	<i>apns</i>	0.903	0.930	0.930	0.931	0.019
		<i>apns-Z</i>	0.947	0.946	0.946	0.992	0.019
V ₄	V ₅	<i>apns</i>	0.891	0.910	0.910	0.911	0.014
		<i>apns-Z</i>	0.925	0.925	0.925	0.981	0.014

[†]Training, [±]Test, [±]Precision, [‡]f-measure, [†]Prediction Error [%]

TABLE IV: Performance comparison of the *assj* project under the *cross-version* scenario

Version		Proj	Performance measurements				
Tr [†]	Ts [◊]		Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
V ₀	V ₁	<i>assj</i>	0.959	0.964	0.964	0.947	0.022
		<i>assj-Z</i>	0.965	0.964	0.964	0.994	0.025
V ₁	V ₂	<i>assj</i>	0.968	0.962	0.962	0.953	0.007
		<i>assj-Z</i>	0.962	0.962	0.962	0.994	0.002
V ₂	V ₃	<i>assj</i>	0.967	0.969	0.969	0.954	0.017
		<i>assj-Z</i>	0.970	0.970	0.969	0.995	0.018
V ₃	V ₄	<i>assj</i>	0.957	0.938	0.938	0.877	0.011
		<i>assj-Z</i>	0.947	0.948	0.947	0.974	0.007
V ₄	V ₅	<i>assj</i>	0.963	0.963	0.962	0.896	0.027
		<i>assj-Z</i>	0.963	0.963	0.962	0.988	0.027

[†]Training, [±]Test, [±]Precision, [‡]f-measure, [†]Prediction Error [%]

TABLE V: Impact of version intervals on the performance for *apns* and *assj* projects under *cross-version* scenario

Proj	Version		Performance measurements					
	Tr [†]	Ts [◊]	Acc [*]	Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
<i>apns</i>	V ₀	V ₁	0.947	0.928	0.947	0.947	0.944	0.029
		V ₂	0.896	0.844	0.896	0.897	0.900	0.039
		V ₃	0.896	0.844	0.896	0.897	0.900	0.039
		V ₄	0.883	0.828	0.883	0.884	0.888	0.052
		V ₅	0.890	0.840	0.890	0.890	0.894	0.055
<i>assj</i>	V ₀	V ₁	0.964	0.959	0.964	0.964	0.947	0.022
		V ₂	0.954	0.958	0.954	0.953	0.940	0.012
		V ₃	0.959	0.956	0.959	0.959	0.938	0.025
		V ₄	0.931	0.949	0.931	0.929	0.857	0.016
		V ₅	0.943	0.948	0.943	0.940	0.849	0.034

[†]Training, [±]Test, ^{*}Accuracy, [±]Precision, [‡]f-measure, [†]Prediction Error [%]

The results of these analyses show for the C project an *AUC* of 0.733 when splitting the data into 80 training data and 20% test data. With the same division, the best result that

TABLE VI: PMT performance for C project (using 24 features)

Version		Performance measurements					
Tr [†]	Ts [◊]	Acc [*]	Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
90%	10%	0.880	0.872	0.880	0.879	0.871	0.036
80%	20%	0.879	0.871	0.879	0.879	0.872	0.036
70%	30%	0.875	0.871	0.875	0.874	0.868	0.031
60%	40%	0.870	0.870	0.870	0.870	0.864	0.027
50%	50%	0.862	0.864	0.862	0.861	0.855	0.025

[†]Training, [±]Test, ^{*}Accuracy, [±]Precision, [‡]f-measure

[†]Prediction Error [%]

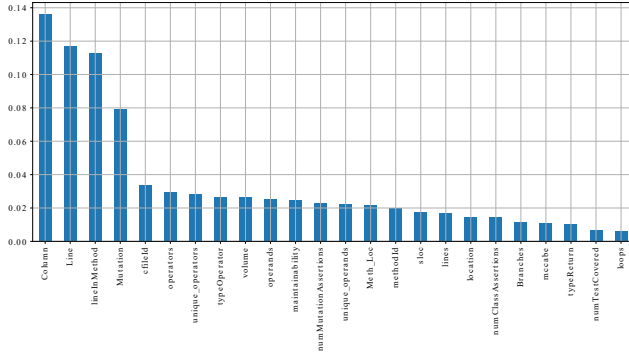


Fig. 3: Feature importance for C project

TABLE VII: Performance measurements of PMT for C project with 5 , 8, 10 and 24 features

Feat [†]	Performance measurements					
	Acc [*]	Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
24	0.879	0.871	0.879	0.879	0.872	0.036
10	0.853	0.854	0.914	0.875	0.916	0.028
8	0.864	0.875	0.916	0.876	0.907	0.036
5	0.872	0.874	0.915	0.894	0.952	0.026

[†]Features, ^{*}Accuracy, [±]Precision, [‡]f-measure

[†]Prediction Error [%]

could be achieved for Java projects with version V_2 , which has on average an AUC of 0.803 value, and the worst result was achieved for version V_5 with an average AUC value of 0.736. This again indicates that using common features gives similar results for C and Java programs.

V. DISCUSSION

The results obtained by Zhang *et al.*[12] were successfully replicated. Under the cross-version application scenario all performance measures are very close to the ones published by Zhang *et al.* [12] for all Java projects. More precisely, F-measure values differ by a maximum of 0.05 and minimum of 0.001, the AUC values differ by a maximum of 0.03 and a minimum of 0.001. The prediction errors are very close too. The effect of the length of version intervals in terms of AUC

TABLE VIII: PMT performance of Java cross-projects scenarios and C project, using common features

Proj	Performance measurements					
	Acc [*]	Pre [±]	Recall	f-meas [‡]	AUC	PE [†]
<i>apns</i>	0.831	0.853	0.831	0.831	0.832	0.024
<i>assj</i>	0.924	0.935	0.924	0.919	0.803	0.039
<i>joda</i>	0.843	0.880	0.843	0.840	0.781	0.026
<i>laffj</i>	0.849	0.857	0.849	0.847	0.823	0.045
<i>lang</i>	0.865	0.900	0.865	0.858	0.717	0.039
<i>msg</i>	0.823	0.839	0.823	0.822	0.815	0.016
<i>uua</i>	0.806	0.784	0.806	0.806	0.806	0.009
<i>vrap</i>	0.858	0.875	0.858	0.855	0.815	0.041
<i>wire</i>	0.870	0.883	0.870	0.868	0.838	0.036
C	0.841	0.860	0.867	0.863	0.916	0.050

^{*}Accuracy, [±]Precision, [‡]f-measure, [†]Prediction Error [%]

values also confirms the accuracy of our replication since our performance values are very close to those presented by Zhang *et al.* [12]. In the cross-project scenario, our results differ from the ones obtained by Zhang *et al.* (cf. Table II). However, when adjusting the default parameter setting in the Python library functions used for building the RF models to the *Weka* default parameters used by Zhang *et al.* in their original study, we obtained similar results, and for some projects even better results.

For the general evaluation of PMT on our C project, Table VI, the prediction error is low, less than 3.6% and all the other performance measures are high, greater than 80%. Based on this finding we believe that PMT can be transferred from Java to the C language, and possibly to other programming languages as well. This finding opens new possibilities for PMT as models trained on one programming language seem to be applicable to projects using other programming languages and still obtain good predictions of mutation scores. PMT greatly reduces the cost of mutant execution and makes MT useful for developers who need to obtain the execution results quickly in exchange for small accuracy losses.

While these initial results are promising, more replications are needed. We only had one (large) C program with associated test cases available for our study.

Several features can be collected for both Java and C projects. Feature selection can be performed to detect what is the smallest feature set that can achieve similar results as it can be obtained using all the features. Less feature collection needs less time. This improves the efficiency of PMT even further. In this research we were able to find five features that can achieve such a result for the studied C project. The same can be done for Java projects. In the C project the most important features are related to location so using those features resulted in good performance of the model and will be investigated more thoroughly in the future.

VI. CONCLUSION

This paper described the transfer of PMT as suggested by Zhang *et al.* [12], [13] for Java projects to a larger C project from industry. We therefore, first, replicated the analyses of the original paper with a slightly different setup (e.g., using Python ML tools). The obtained results confirmed the findings reported by Zhang *et al.* and the suitability of our setup. We then applied PMT to a safety-critical embedded system implemented in C, for which we were able to achieve very good performance. Our results demonstrate that PMT is also useful in the context of C projects, and the approach can possibly be extended to other programming languages as well.

ACKNOWLEDGEMENT

The authors have been supported by the Austrian ministries BMVIT and BMDW, and the Province of Upper Austria under the COMET (Competence Centers for Excellent Technologies) Programme managed by FFG, as well as the Estonian Center of Excellence in ICT research (EXCITE).

REFERENCES

- [1] A. Memon, Zebao Gao, Bao Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track*, ser. ICSE-SEIP'17, Buenos Aires, Argentina, May 2017, pp. 233–242.
- [2] S.-W. Kim, J. A. Clark, and J. A. McDermid, "Investigating the effectiveness of object-oriented testing strategies using the mutation method," *Software Testing, Verification and Reliability*, vol. 11, no. 4, pp. 207–225, 2001.
- [3] R. Gopinath, C. Jensen, and A. Groce, "Code coverage for suite evaluation by developers," in *36th International Conference on Software Engineering*, Hyderabad, India, 2014, pp. 72–82.
- [4] M. Alenezi, M. Akour, A. Hussien, and M. Z. Al-Saad, "Test suite effectiveness: An indicator for open source software quality," in *2nd International Conference on Open Source Software Computing*, ser. OSSCOM, Beirut, Lebanon, Dec. 2016, pp. 1–5.
- [5] J. H. Andrews, L. C. Briand, and Y. Labiche, "Is mutation an appropriate tool for testing experiments? [software testing]," in *Proceedings of 27th International Conference on Software Engineering*, ser. ICSE'05, Saint Louis, MO, USA, USA, May 2005, pp. 402–411.
- [6] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," in *IEEE Transactions on Software Engineering*, vol. 37, Sep. 2011, pp. 649–678.
- [7] S. Vercammen, S. Demeyer, M. Borg, and S. Eldh, "Speeding up mutation testing via the cloud: Lessons learned for further optimisations," in *ESEM*, 2018, pp. 26–1.
- [8] J. Zhang, M. Zhu, D. Hao, and L. Zhang, "An empirical study on the scalability of selective mutation testing," in *25th Int. Symposium on Software Reliability Engineering*, Naples, Italy, 2014, pp. 277–287.
- [9] M. Papadakis and N. Maleveris, "An empirical evaluation of the first and second order mutation testing strategies," in *3th IEEE International Conference on Software Testing, Verification, and Validation Workshops*, ser. ICSTW'10, Paris, France, May 2010, pp. 90–99.
- [10] P. Ammann, M. E. Delamaro, and J. Offutt, "Establishing theoretical minimal sets of mutants," in *7th IEEE International Conference on Software Testing, Verification and Validation*, ser. ICST'14, Cleveland, OH, USA, Mar. 2014, pp. 21–30.
- [11] B. Kurtz, P. Ammann, and J. Offutt, "Static analysis of mutant subsumption," in *IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops*, ser. ICSTW'15, Graz, Austria, Apr. 2015, pp. 1–10.
- [12] J. Zhang, Z. Wang, L. Zhang, D. Hao, L. Zang, S. Cheng, and L. Zhang, "Predictive mutation testing," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ser. ISSTA'16, Saarbrücken, Germany, 2016, pp. 342–353.
- [13] J. Zhang, L. Zhang, M. Harman, D. Hao, Y. Jia, and L. Zhang, "Predictive mutation testing," in *IEEE Transactions on Software Engineering*, vol. 45, Sep. 2019, pp. 898–918.
- [14] D. Mao, L. Chen, and L. Zhang, "An extensive study on cross-project predictive mutation testing," in *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, Apr. 2019, pp. 160–171.
- [15] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*, vol. 112, Elsevier, 2019, pp. 275–378.
- [16] A. Brillout, N. He, M. Mazzucchi, D. Kroening, M. Purandare, P. Rümmer, and G. Weissenbacher, "Mutation-based test case generation for simulink models," in *Formal Methods for Components and Objects*, Berlin, Heidelberg, 2010, pp. 208–227.
- [17] L. Zhang, T. Xie, L. Zhang, N. Tillmann, J. de Halleux, and H. Mei, "Test generation via dynamic symbolic execution for mutation testing," in *2010 IEEE International Conference on Software Maintenance*, Timisoara, Romania, Sep. 2010, pp. 1–10.
- [18] M. Papadakis, D. Shin, S. Yoo, and D. Bae, "Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults," in *IEEE/ACM 40th International Conference on Software Engineering*, ser. ICSE, Gothenburg, Sweden, May 2018, pp. 537–548.
- [19] Y.-M. Choi and D.-J. Lim, "Model-based test suite generation using mutation analysis for fault localization," in *Applied Sciences*, vol. 9, 2019.
- [20] N. Gupta, A. Sharma, and M. K. Pachariya, "A novel approach for mutant diversity-based fault localization: Dam-fl," in *International Journal of Computers and Applications*, vol. 0, 2019, pp. 1–10.
- [21] Y.-S. Ma, J. Offutt, and Y.-R. Kwon, "Mujava: A mutation system for java," in *Proceedings of the 28th international conference on Software engineering*, ACM, 2006, pp. 827–830.
- [22] R. Just, "The major mutation framework: Efficient and scalable mutation analysis for java," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA'14, San Jose, CA, USA, 2014, pp. 433–436.
- [23] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "Pit: A practical mutation testing tool for java," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ACM, 2016, pp. 449–452.
- [24] Y. Jia and M. Harman, "Milu: A customizable, runtime-optimized higher order mutation testing tool for the full c language," in *Testing: Academic & Industrial Conference-Practice and Research Techniques (taic part 2008)*, IEEE, 2008, pp. 94–98.
- [25] J. Mořucha and B. Rossi, "Is mutation testing ready to be adopted industry-wide?" In *International Conference on Product-Focused Software Process Improvement*, Springer, 2016, pp. 217–232.
- [26] R. Ramler, T. Wetzlmaier, and C. Klammer, "An empirical study on the application of mutation testing for a safety-critical industrial software system," in *Proceedings of the Symposium on Applied Computing*, ser. SAC '17, Marrakech, Morocco, 2017, pp. 1401–1408.
- [27] A. V. Pizzolo, F. C. Ferrari, J. Offutt, L. Fernandes, and M. Ribeiro, "A systematic literature review of techniques and metrics to reduce the cost of mutation testing," *Journal of Systems and Software*, 2019.
- [28] J. M. Voas, "Pie: A dynamic failure-based technique," in *IEEE Transactions on Software Engineering*, vol. 18, Aug. 1992, pp. 717–727.
- [29] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "Pit: A practical mutation testing tool for java (demo)," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ser. ISSTA'16, Saarland University, Saarbrücken, Germany, 2016, pp. 449–452.
- [30] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The WEKA data mining software: An update," in *Special Int. Group on Knowledge Discovery and Data Mining*, ser. SIGKDD'09, vol. 11, Paris, France, 2009, pp. 10–18.
- [31] C. M. Bishop, *Pattern recognition and machine learning*. Springer, 2006.
- [32] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," in *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.